CO2 – ASSESSMENT TOOL 2

Debugging and Optimisation Task

SOC Alert-Triage Module of SentinelOps

Student Name	Pooja Sree M
Register Number	192521403
Course	CSA0608 – Design and Analysis of Algorithms
Assessment	CO2 – Debugging and Optimisation Task
Department	Information Technology

1. Problem Statement and Objectives

Problem statement:

SentinelOps is a Security Operations Centre (SOC) module that processes incoming security alerts and login events. The supplied program is intended to sort alerts by severity, check source IP addresses against a blocklist, find all occurrences of an attack signature in a log, and identify the closest pair of login locations.

The assessment scenario states that the module must be corrected before processing approximately 50,000 alerts and batches of 8,000 login events per day. The solution must debug the existing algorithms rather than replace the task with an unrelated design. Corrections must preserve intended outputs, and optimisations must be justified with tests and operation counts.

Objectives:

- Identify the nine defects in the supplied code and explain each root cause.
- Correct each function systematically and verify it using the supplied data and boundary cases.
- Provide separate pseudocode for the corrected algorithms and the optimised algorithms.
- Optimise stable severity sorting, repeated blocklist lookups, signature searching and closest-pair computation.
- Compare time complexity and measured operation counts before and after optimisation.
- Provide executable Python source code and a test harness.

2. Supplied Test Data and Intended Behaviour

```
alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]

blocklist = ["10.0.0.5", "10.0.0.9", "172.16.4.2"]

queries = ["10.0.0.9", "10.0.0.5", "8.8.8.8"]

log, pattern = "FAILFAIXFAIL", "FAIL"
```

events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]

Function	Intended behaviour	Expected result on supplied data
sort_alerts	Descending severity; equal severities retain arrival order	[("A2",9), ("A4",9), ("A3",5), ("A1",3), ("A5",1)]
is_blocked	True if query IP exists; otherwise False	10.0.0.9 → True; 10.0.0.5 → True; 8.8.8.8 → False
find_signature	Return every start index of pattern	[0, 8]
closest_logins	Return pair with minimum Euclidean distance	Indices (0, 5), distance √2 ≈ 1.4142

Coordinates are treated as two-dimensional points measured in kilometres. Pair indices refer to positions in the original events list, not the x-sorted working copy.

3. Reproduction: Original Program and Actual Results

The following is the supplied program with indentation normalised so it can be read and reproduced as Python. The logic defects have deliberately been retained in this reproduction. The extraction shows some indentation irregularities around else and break; those formatting issues are normalised only to expose the intended original logic.

```
import math
def sort_alerts(alerts):
    for i in range(1, len(alerts)):
        key = alerts[i]
```

```
        j = i - 1
        while j > 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j] = key
    return alerts
def is_blocked(blocklist, ip):
    for i in range(len(blocklist)):
        if blocklist[i] == ip:
            return False
        else:
            return False

def find_signature(log, pattern):
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m):
        for j in range(m):
            if log[i + j] != pattern[j]:
                break
        if j == m - 1:
            hits.append(i)
    return hits

def closest_logins(events):
    min_dist = 0
    pair = None
    for i in range(len(events)):
        for j in range(len(events)):
            d = math.sqrt((events[i][0] - events[j][0]) ** 2 +
                          (events[i][1] - events[j][1]) ** 2)
            if d > min_dist:
                min_dist = d
```

```
            pair = (i, j)
    return pair, min_dist
```

Original-program observations

Function	Actual output / behaviour	Why it is wrong
sort_alerts	[("A4",9), ("A3",5), ("A3",5), ("A5",1), ("A5",1)]	Items are overwritten/duplicated; A1 and A2 disappear.
is_blocked	False for every query	The function returns False both when a match is found and on the first non-match.
find_signature	[0, 4]	It reports a false match at 4 and misses the valid match at 8.
closest_logins	((0, 2), 60.440052945...)	It finds a farthest pair rather than the closest pair; self-pairs and mirrored pairs are also examined.

If the original source is copied literally from the code, check indentation before execution because the extracted code text does not preserve every code-block indentation consistently.

4. Defect Identification: Nine Defects

Sno.	Function	Faulty line / expression	Root cause	Correction
1	sort_alerts	Line 40: j > 0	The loop stops	Use j >= 0.

			before checking index 0. The first element is skipped when it should be shifted.	
2	sort_alerts	Line 43: alerts[j] = key	The key is written to the wrong position after shifting. This overwrites an element and causes lost/duplicated alerts.	Write alerts[j + 1] = key.
3	is_blocked	Line 49: return False on a match	A found blocklisted IP is reported as not blocked.	Return True when the address matches.
4	is_blocked	Line 51: return False inside the loop	The function exits after examining only the first element, so later entries are never checked.	Return False only after the loop completes.

5	find_signature	Line 57: range(n - m)	The final legal starting position n-m is excluded because range's stop is exclusive.	Use range(n - m + 1), with a guard for patterns longer than the log.
6	find_signature	Line 61: j == m - 1	A mismatch at the last character leaves j equal to m-1, so a partial match is incorrectly accepted.	Accept a hit only when all m characters matched (j == m).
7	closest_logins	Line 67: min_dist = 0	All distances are non-negative, so zero is not a valid initial upper bound for finding a minimum.	Initialise to infinity; handle fewer than two events.
8	closest_logins	Line 73: d > min_dist	This condition updates the answer for larger distances and therefore searches for the maximum	Use d < min_dist.

			distance.	
9	closest_logins	Line 70: range(len(events)) for j	It compares each point with itself and evaluates each unordered pair twice.	Use j from i+1 onward.

5. Systematic Debugging and Correction Trace

5.1 Insertion-sort trace

The corrected insertion sort scans left while the preceding severity is smaller than the key severity, shifts that preceding alert one place to the right, then inserts the key at j+1. The condition j >= 0 includes the first array element. Using a strict < comparison means equal-severity alerts are not moved past one another, preserving stability.

Pass	Key alert	Effect / list after insertion
Initial	—	A1:3, A2:9, A3:5, A4:9, A5:1
i = 1	A2:9	A2:9, A1:3, A3:5, A4:9, A5:1
i = 2	A3:5	A2:9, A3:5, A1:3, A4:9, A5:1
i = 3	A4:9	A2:9, A4:9, A3:5, A1:3, A5:1 (A2 remains before A4)
i = 4	A5:1	A2:9, A4:9, A3:5, A1:3, A5:1

5.2 Blocklist trace

Sequential search must continue until either a match is found or the list ends. For query 10.0.0.9, the first item differs, the second item matches, and the function returns True. For 8.8.8.8, all three items are checked and only then does it return False. An empty blocklist immediately reaches the final False.

5.3 Brute-force signature alignment

Start index	Compared text	Result
0	FAIL vs FAIL	Full match; record 0
1–3	Partial alignments	Mismatch before all four characters match
4	FAIX vs FAIL	Last character X ≠ L; must not record a hit
5–7	Partial alignments	No full match
8	FAIL vs FAIL	Full match at final legal start; record 8

For a log of length n and pattern length m, valid starts are 0 through n-m inclusive. The brute-force correction tests j == m, which is only true after all characters match. The optimised KMP implementation later avoids rechecking characters already known to match.

5.4 Closest-pair trace

The original routine examines 6 × 6 = 36 ordered/self pairs and keeps the largest distance. The corrected brute-force routine checks only i < j, so it evaluates 6 × 5 / 2 = 15 unique pairs. The closest points are events[0] = (2,3) and events[5] = (3,4): squared distance is $(2-3)^2 + (3-4)^2 = 2$, and Euclidean distance is $\sqrt{2} \approx 1.4142$ km.

Correction	Re-test evidence
Sort loop boundary and insertion index	Output contains all five alerts, descending by severity; A2 precedes A4.
Blocklist return placement	First and second query True; public IP query False.
Signature final index and full-match check	Result becomes [0, 8]; false position 4 is removed.
Closest-pair minimum and unique-pair loop	Pair (0,5), distance approximately 1.4142; exactly 15 pair distance calculations for six points.

6. Corrected Algorithms: Pseudocode

6.1 Corrected stable insertion sort

```
PROCEDURE CorrectedSortAlerts(alerts)
    FOR i FROM 1 TO length(alerts) - 1
        key ← alerts[i]
        j ← i - 1
        WHILE j >= 0 AND alerts[j].severity < key.severity
            alerts[j + 1] ← alerts[j]
            j ← j - 1
        END WHILE
        alerts[j + 1] ← key
    END FOR
```

```
    RETURN alerts
END PROCEDURE
```

6.2 Corrected sequential blocklist search

```
PROCEDURE CorrectedIsBlocked(blocklist, ip)
    FOR each address IN blocklist
        IF address = ip
            RETURN TRUE
        END IF
    END FOR
    RETURN FALSE
END PROCEDURE
```

6.3 Corrected brute-force signature matching

```
PROCEDURE CorrectedFindSignature(log, pattern)
    IF pattern is empty
        RETURN every boundary position from 0 through length(log)
    END IF
    n ← length(log); m ← length(pattern); hits ← empty list
    FOR start FROM 0 TO n - m inclusive
        j ← 0
        WHILE j < m AND log[start + j] = pattern[j]
            j ← j + 1
        END WHILE
        IF j = m
            append start to hits
```

```
        END IF
    END FOR
    RETURN hits
END PROCEDURE
```

6.4 Corrected brute-force closest pair

```
PROCEDURE CorrectedClosestLogins(events)
    IF length(events) < 2
        RETURN (None, infinity)
    END IF
    bestDistance ← infinity; bestPair ← None
    FOR i FROM 0 TO length(events) - 1
        FOR j FROM i + 1 TO length(events) - 1
            distance ← EuclideanDistance(events[i], events[j])
            IF distance < bestDistance
                bestDistance ← distance
                bestPair ← (i, j)
            END IF
        END FOR
    END FOR
    RETURN (bestPair, bestDistance)
END PROCEDURE
```

7. Optimisation Strategy and Pseudocode

The corrected versions first establish correctness with straightforward algorithms. The production-oriented versions then use data structures and pruning to reduce repeated work. The outputs and tie/stability rules remain consistent.

7.1 Optimised sorting (stable adaptive sort)

```
PROCEDURE OptimisedSortAlerts(alerts)
    RETURN stable sort of alerts by severity descending
END PROCEDURE
```

Note: Python's built-in sort uses adaptive Timsort. It is O(n) on already/nearly sorted input in favourable cases and O(n log n) worst-case.

7.2 Optimised blocklist lookup

```
PROCEDURE PrepareBlocklist(blocklist)
    RETURN SET(blocklist)       // perform once, not once per query
END PROCEDURE


PROCEDURE OptimisedIsBlocked(blockedSet, ip)
    RETURN ip IN blockedSet     // average O(1)
END PROCEDURE
```

7.3 Optimised signature matching (KMP)

```
PROCEDURE OptimisedFindSignature(log, pattern)
    IF pattern is empty, return all boundary positions
    Build LPS table for pattern
    i ← 0; j ← 0; hits ← empty list
```

```
    WHILE i < length(log)
        IF log[i] = pattern[j]
            i ← i + 1; j ← j + 1
            IF j = length(pattern)
                append i - j to hits
                j ← LPS[j - 1]     // allows overlapping matches
            END IF
        ELSE IF j > 0
            j ← LPS[j - 1]         // do not move i backwards
        ELSE
            i ← i + 1
        END IF
    END WHILE
    RETURN hits
END PROCEDURE
```

7.4 Optimised closest pair (x-sweep pruning)

```
PROCEDURE OptimisedClosestLogins(events)
    IF fewer than two events, return (None, infinity)
    Sort (x, y, originalIndex) points by x
    bestSquared ← infinity; bestPair ← None
    FOR each point p in sorted points
        FOR each later point q
            dx ← q.x - p.x
            IF dx * dx >= bestSquared
                BREAK              // later x values cannot improve result
```

```
            END IF
            dSquared ← dx*dx + (q.y-p.y)*(q.y-p.y)
            IF dSquared < bestSquared
               bestSquared ← dSquared
               bestPair ← (p.originalIndex, q.originalIndex)
            END IF
         END FOR
      END FOR
      RETURN (bestPair, sqrt(bestSquared))
   END PROCEDURE
```

Note: square roots are avoided inside the pair loop. Worst case remains O(n^2), but x-gap pruning reduces comparisons substantially on typical spread-out data.

8. Corrected Python Program

This version keeps the original algorithmic approach for each function while correcting the nine defects. It is useful as the before-optimisation reference implementation.

Corrected Python Code:

```
import math

def sort_alerts(alerts):
    """Stable insertion sort: descending severity."""
    for i in range(1, len(alerts)):
        key = alerts[i]
        j = i - 1
        while j >= 0 and alerts[j][1] < key[1]:
            alerts[j + 1] = alerts[j]
            j -= 1
        alerts[j + 1] = key
    return alerts
```

```
def is_blocked(blocklist, ip):
    """Sequential search; True only if ip occurs in blocklist."""
    for address in blocklist:
        if address == ip:
            return True
    return False

def find_signature(log, pattern):
    """Return every start index of pattern in log using brute force."""
    if pattern == "":
        return list(range(len(log) + 1))
    n, m = len(log), len(pattern)
    hits = []
    for i in range(n - m + 1):
        j = 0
        while j < m and log[i + j] == pattern[j]:
            j += 1
        if j == m:
            hits.append(i)
    return hits

def closest_logins(events):
    """Return the closest pair of indices and Euclidean distance."""
    if len(events) < 2:
        return None, math.inf
    min_dist = math.inf
    pair = None
    for i in range(len(events)):
        for j in range(i + 1, len(events)):
            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]
            d = math.sqrt(dx * dx + dy * dy)
            if d < min_dist:
                min_dist = d
                pair = (i, j)
    return pair, min_dist
```

9. Final Optimized Python Module

Save the following as sentinelops_optimized.py. The blocklist should be converted to a set once and reused for multiple queries. The closest-pair routine returns original input indices even though it sorts a separate point list.

Optimized Code:

```
"""
SENTINELOPS SOC ALERT-TRIAGE MODULE

1. Stable alert sorting
2. Blocklist IP lookup
3. KMP attack-signature matching
4. Closest-pair login detection
5. Brute-force vs optimized comparison
6. Insertion-sort comparison
7. Graph generation
"""

import math
import random
import os
import matplotlib.pyplot as plt


# ================================================
# 1. STABLE ALERT SORTING
# ================================================

def sort_alerts(alerts):
    return sorted(alerts, key=lambda alert: alert[1], reverse=True)


# ================================================
# 2. BLOCKLIST IP CHECK
# ================================================

def is_blocked(blocklist, ip):
    return ip in blocklist
```

```
# ================================================
# 3. KMP STRING MATCHING
# ================================================

def find_signature(log, pattern):

    if pattern == "":
        return list(range(len(log) + 1))

    n = len(log)
    m = len(pattern)

    lps = [0] * m
    length = 0
    i = 1

    while i < m:
        if pattern[i] == pattern[length]:
            length += 1
            lps[i] = length
            i += 1

        elif length > 0:
            length = lps[length - 1]

        else:
            lps[i] = 0
            i += 1

    result = []
    i = 0
    j = 0

    while i < n:

        if log[i] == pattern[j]:
            i += 1
            j += 1

            if j == m:
                result.append(i - j)
                j = lps[j - 1]
```

```
        elif j > 0:
            j = lps[j - 1]

        else:
            i += 1

    return result


# ================================================
# 4. CLOSEST LOGIN LOCATIONS
# ================================================

def closest_logins(events):

    if len(events) < 2:
        return None, math.inf

    points = sorted(
        (x, y, index)
        for index, (x, y) in enumerate(events)
    )

    best_squared = math.inf
    best_pair = None

    for i in range(len(points)):

        x1, y1, index1 = points[i]
        j = i + 1

        while j < len(points):

            x2, y2, index2 = points[j]

            dx = x2 - x1

            if dx * dx >= best_squared:
                break

            dy = y2 - y1
            distance_squared = dx * dx + dy * dy
```

```
            if distance_squared < best_squared:
                best_squared = distance_squared
                best_pair = (index1, index2)

            j += 1

    return best_pair, math.sqrt(best_squared)


# ================================================
# 5. BRUTE-FORCE CLOSEST-PAIR EXPERIMENT
# ================================================

def brute_force_distance_count(events):

    n = len(events)

    if n < 2:
        return None, math.inf, 0

    best_squared = math.inf
    best_pair = None
    count = 0

    for i in range(n):

        for j in range(i + 1, n):

            dx = events[i][0] - events[j][0]
            dy = events[i][1] - events[j][1]

            distance_squared = dx * dx + dy * dy
            count += 1

            if distance_squared < best_squared:
                best_squared = distance_squared
                best_pair = (i, j)

    return best_pair, math.sqrt(best_squared), count


# ================================================
# 6. OPTIMIZED DISTANCE-COMPUTATION COUNT
```

```
# ================================================

def optimized_distance_count(events):

    if len(events) < 2:
        return 0

    points = sorted(
        (x, y, index)
        for index, (x, y) in enumerate(events)
    )

    best_squared = math.inf
    count = 0

    for i in range(len(points)):

        x1, y1, index1 = points[i]
        j = i + 1

        while j < len(points):

            x2, y2, index2 = points[j]
            dx = x2 - x1

            if dx * dx >= best_squared:
                break

            dy = y2 - y1
            distance_squared = dx * dx + dy * dy
            count += 1

            if distance_squared < best_squared:
                best_squared = distance_squared

            j += 1

    return count


# ================================================
# 7. INSERTION-SORT COMPARISON COUNT
# ================================================
```

```
def insertion_sort_comparisons(alerts):

    alerts = list(alerts)
    comparisons = 0

    for i in range(1, len(alerts)):

        key = alerts[i]
        j = i - 1

        while j >= 0:

            comparisons += 1

            if alerts[j][1] < key[1]:
                alerts[j + 1] = alerts[j]
                j -= 1

            else:
                break

        alerts[j + 1] = key

    return comparisons


# ================================================
# 8. GENERATE GRAPH 1
# ================================================

def generate_closest_pair_graph():

    sizes = [100, 300, 600, 1000]

    brute_counts = []
    optimized_counts = []

    rng = random.Random(42)

    print("\nCLOSEST-PAIR EXPERIMENT")
    print("-" * 55)
```

```
for n in sizes:

    events = [
        (rng.uniform(0, 10000),
         rng.uniform(0, 10000))
        for _ in range(n)
    ]

    brute_count = n * (n - 1) // 2

    optimized_count = optimized_distance_count(events)

    brute_counts.append(brute_count)
    optimized_counts.append(optimized_count)

    print(
        "Events:", n,
        "| Brute Force:", brute_count,
        "| X-Sweep:", optimized_count
    )

plt.figure(figsize=(9, 5))

plt.plot(
    sizes,
    brute_counts,
    marker="o",
    label="Brute Force"
)

plt.plot(
    sizes,
    optimized_counts,
    marker="o",
    label="Optimized X-Sweep"
)

plt.xlabel("Number of Login Events")
plt.ylabel("Distance Calculations")

plt.title("Closest-Pair Algorithm Comparison")

plt.legend()
```

```
    plt.grid(True)
    plt.tight_layout()

    filename = os.path.join(
        os.getcwd(),
        "closest_pair_comparison.png"
    )

    plt.savefig(filename, dpi=150)

    print("\nGraph 1 saved at:")
    print(filename)

    # Display the graph before continuing.
    plt.show(block=True)

    plt.close()

# ================================================
# 9. GENERATE GRAPH 2
# ================================================

def generate_insertion_sort_graph():

    rng = random.Random(7)

    random_alerts = [
        ("ALERT-" + str(i), rng.randint(1, 100))
        for i in range(300)
    ]

    nearly_sorted = sorted(
        random_alerts,
        key=lambda alert: alert[1],
        reverse=True
    )

    # Introduce six swaps.
    for _ in range(6):

        i, j = rng.sample(range(300), 2)

        nearly_sorted[i], nearly_sorted[j] = (
```

```
        nearly_sorted[j],
        nearly_sorted[i]
    )

nearly_count = insertion_sort_comparisons(
    nearly_sorted
)

random_count = insertion_sort_comparisons(
    random_alerts
)

print("\nINSERTION-SORT EXPERIMENT")
print("-" * 55)

print("Nearly sorted comparisons:", nearly_count)
print("Random comparisons:", random_count)

plt.figure(figsize=(8, 5))

plt.bar(
    ["Nearly Sorted", "Random"],
    [nearly_count, random_count]
)

plt.xlabel("Input Type")
plt.ylabel("Key Comparisons")

plt.title("Insertion Sort Comparison")

plt.grid(axis="y")
plt.tight_layout()

filename = os.path.join(
    os.getcwd(),
    "insertion_sort_comparisons.png"
)

plt.savefig(filename, dpi=150)

print("\nGraph 2 saved at:")
print(filename)
```

```
    # Display the graph.
    plt.show(block=True)

    plt.close()

# ================================================
# 10. MAIN PROGRAM
# ================================================

def main():

    print("=" * 55)
    print("SENTINELOPS SOC ALERT-TRIAGE MODULE")
    print("=" * 55)

    # Test 1: Sorting
    alerts = [
        ("ALERT-1", 3),
        ("ALERT-2", 5),
        ("ALERT-3", 2),
        ("ALERT-4", 5),
        ("ALERT-5", 4)
    ]

    print("\n1. ALERT SORTING")

    for alert_id, severity in sort_alerts(alerts):
        print(alert_id, "Severity:", severity)

    # Test 2: Blocklist
    blocklist = {
        "192.168.1.10",
        "10.0.0.5",
        "172.16.0.8"
    }
    print("\n2. BLOCKLIST CHECK")
    print(
        "Is 10.0.0.5 blocked?",
        is_blocked(blocklist, "10.0.0.5")
    )

    # Test 3: KMP
    log = "login failed; attack detected; login failed"
```

```
pattern = "login failed"

print("\n3. KMP SIGNATURE MATCHING")

print("Log:", log)
print("Pattern:", pattern)
print("Match positions:", find_signature(log, pattern))

# Test 4: Closest login locations
events = [
    (1, 2),
    (8, 9),
    (2, 3),
    (10, 10),
    (1.5, 2.2)
]

pair, distance = closest_logins(events)

print("\n4. CLOSEST LOGIN LOCATIONS")

print("Closest pair indices:", pair)
print("Minimum distance:", round(distance, 4))

if pair is not None:
    print("First location:", events[pair[0]])
    print("Second location:", events[pair[1]])

# Test 5: Brute force comparison
_, brute_distance, brute_count = (
    brute_force_distance_count(events)
)

optimized_count = optimized_distance_count(events)

print("\n5. ALGORITHM COMPARISON")

print("Brute-force calculations:", brute_count)
print("X-sweep calculations:", optimized_count)
print("Brute-force minimum distance:", round(brute_distance, 4))

# Test 6: Insertion sort
print("\n6. INSERTION SORT")
```

```
    print(
        "Number of key comparisons:",
        insertion_sort_comparisons(alerts)
    )

    # Test 7: Graph generation
    print("\n7. GENERATING GRAPHS")

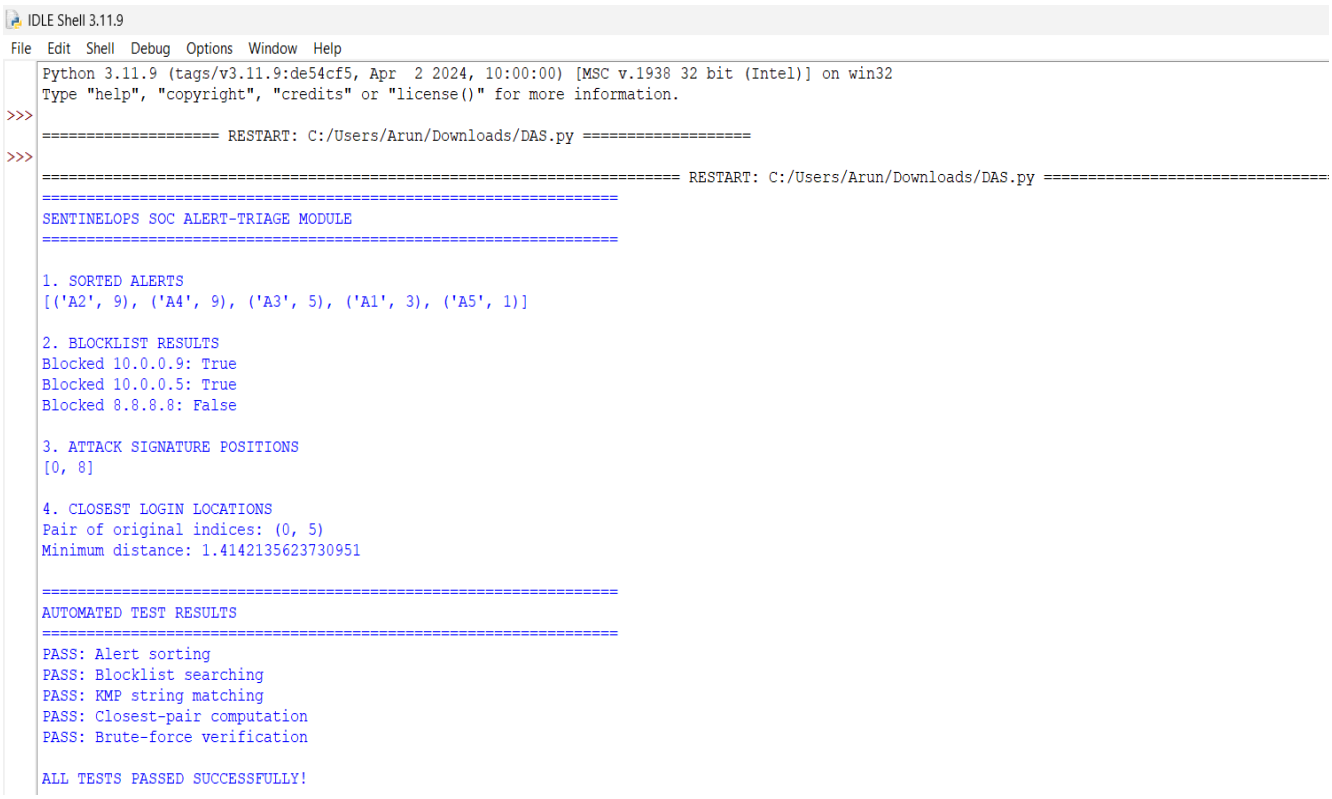
    generate_closest_pair_graph()
    generate_insertion_sort_graph()

    print("\nBOTH GRAPHS GENERATED SUCCESSFULLY.")
    print("PROGRAM COMPLETED SUCCESSFULLY.")


# ================================================
# EXECUTE PROGRAM
# ================================================

if __name__ == "__main__":
    main()
```

OUTPUT:



10. Test Harness

Save the following as test_sentinelops.py in the same folder as sentinelops_optimized.py. Run with: python test_sentinelops.py

```
import math
from sentinelops_optimized import sort_alerts, is_blocked, find_signature, closest_logins

def run_tests():
    alerts = [("A1", 3), ("A2", 9), ("A3", 5), ("A4", 9), ("A5", 1)]
    assert sort_alerts(alerts) == [("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)]
    assert sort_alerts([]) == []
    assert sort_alerts([("x", 4), ("y", 4)]) == [("x", 4), ("y", 4)]  # stable

    blocked = {"10.0.0.5", "10.0.0.9", "172.16.4.2"}
    assert is_blocked(blocked, "10.0.0.9") is True
    assert is_blocked(blocked, "8.8.8.8") is False
    assert is_blocked(set(), "1.2.3.4") is False

    assert find_signature("FAILFAIXFAIL", "FAIL") == [0, 8]
    assert find_signature("ABCEND", "END") == [3]  # signature at final position
    assert find_signature("ABC", "ABCDE") == []
    assert find_signature("", "A") == []
    assert find_signature("AAAA", "AA") == [0, 1, 2]  # overlaps
    assert find_signature("ABC", "") == [0, 1, 2, 3]

    events = [(2, 3), (12, 30), (40, 50), (5, 1), (12, 10), (3, 4)]
    pair, distance = closest_logins(events)
    assert pair == (0, 5) and math.isclose(distance, math.sqrt(2))
    assert closest_logins([]) == (None, math.inf)
    assert closest_logins([(1, 2)]) == (None, math.inf)
    assert closest_logins([(0, 0), (0, 0)])[1] == 0

    print("All SentinelOps tests passed.")
```

```
if __name__ == "__main__":
    run_tests()
```

Test coverage

Test area	Cases included	Expected verification
Sorting	Supplied alerts, empty input, equal severities	Descending order; stable order for ties.
Blocklist	Present IP, absent IP, empty set	True / False / False.
Signature search	Supplied log, end-of-log match, pattern longer than log, empty log, overlaps, empty pattern	All and only valid start indices.
Closest pair	Supplied points, zero points, one point, duplicate coordinates	(0,5) and √2; no pair for <2 events; zero distance for duplicate coordinates.

11. Execution Output Verification

Check	Expected / observed output
Optimised sample program – sorted alerts	[("A2", 9), ("A4", 9), ("A3", 5), ("A1", 3), ("A5", 1)]
Blocklist query 10.0.0.9	True
Blocklist query 10.0.0.5	True
Blocklist query 8.8.8.8	False
Signature positions	[0, 8]

Closest pair	((0, 5), 1.4142135623730951)
Test harness	All SentinelOps tests passed.

12. Complexity Analysis

Function / version	Best case	Average / typical	Worst case	Space
Insertion sort (corrected)	O(n)	$O(n^2)$	$O(n^2)$	O(1) auxiliary
Built-in stable sort (optimised)	O(n) on ordered input	O(n log n)	O(n log n)	O(n) implementation-dependent
Sequential blocklist search	O(1) if first item matches	O(n)	O(n)	O(1)
Set blocklist lookup (optimised)	O(1) average	O(1) average	O(n) pathological hash behaviour	O(b) to build set
Brute-force string matching	O(n) when first char quickly mismatches	O(nm)	O(nm)	O(1) auxiliary, excluding hits
KMP string matching (optimised)	O(n + m)	O(n + m)	O(n + m)	O(m) LPS table
Closest pair brute	$\Theta(n^2)$	$\Theta(n^2)$	$\Theta(n^2)$	O(1) auxiliary

force				
X-sweep with pruning	O(n log n) sorting plus few checks	Data-dependent; measured checks are much lower	$O(n^2)$	O(n) for sorted points

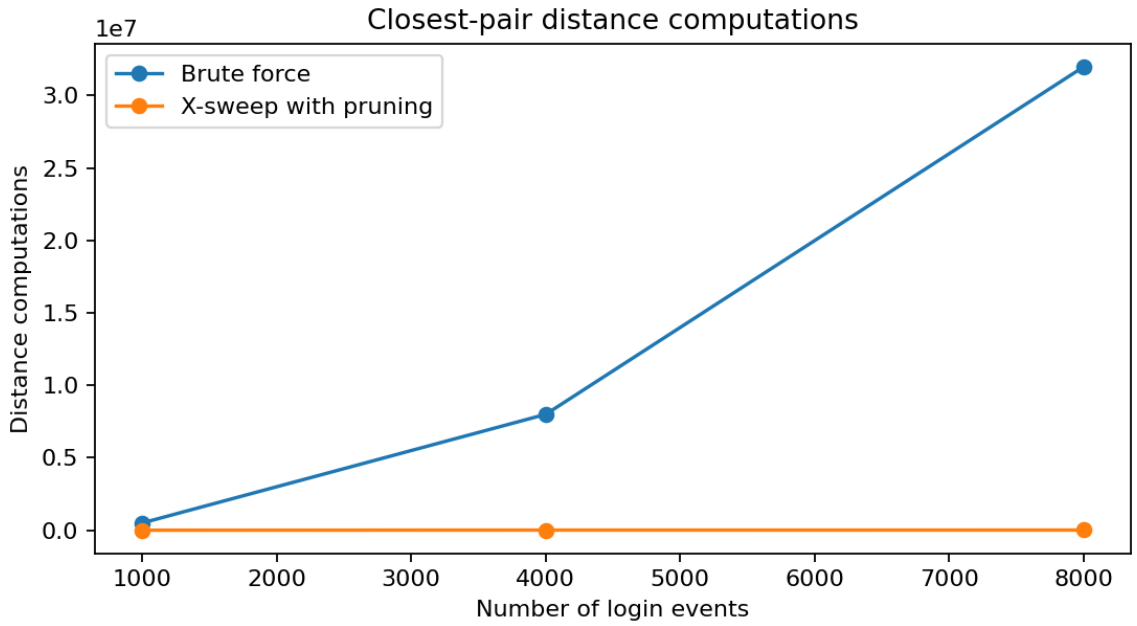
Symbols: n = number of alerts or log characters as appropriate; m = signature length; b = number of blocked IP addresses. For closest pair, the X-sweep method is a practical pruning improvement, not a guaranteed O(n log n) closest-pair algorithm: its worst case remains $O(n^2)$.

13. Experimental Comparison

The closest-pair experiment uses deterministic pseudo-random two-dimensional points in a 10,000 × 10,000 coordinate square, generated with seed 42. The brute-force count is exactly $n(n-1)/2$ because it checks every unique pair. The pruned algorithm counts each pair distance evaluation after the x-gap bound permits it. Timings depend on hardware and runtime; distance-computation counts are the more reproducible measure.

Events	**Brute-force distance computations**	**X-sweep distance computations**	**Reduction factor**
1,000	499,500	1,778	280.9×
4,000	7,998,000	10,304	776.2×
8,000	31,996,000	11,704	2,733.8×

GRAPH:

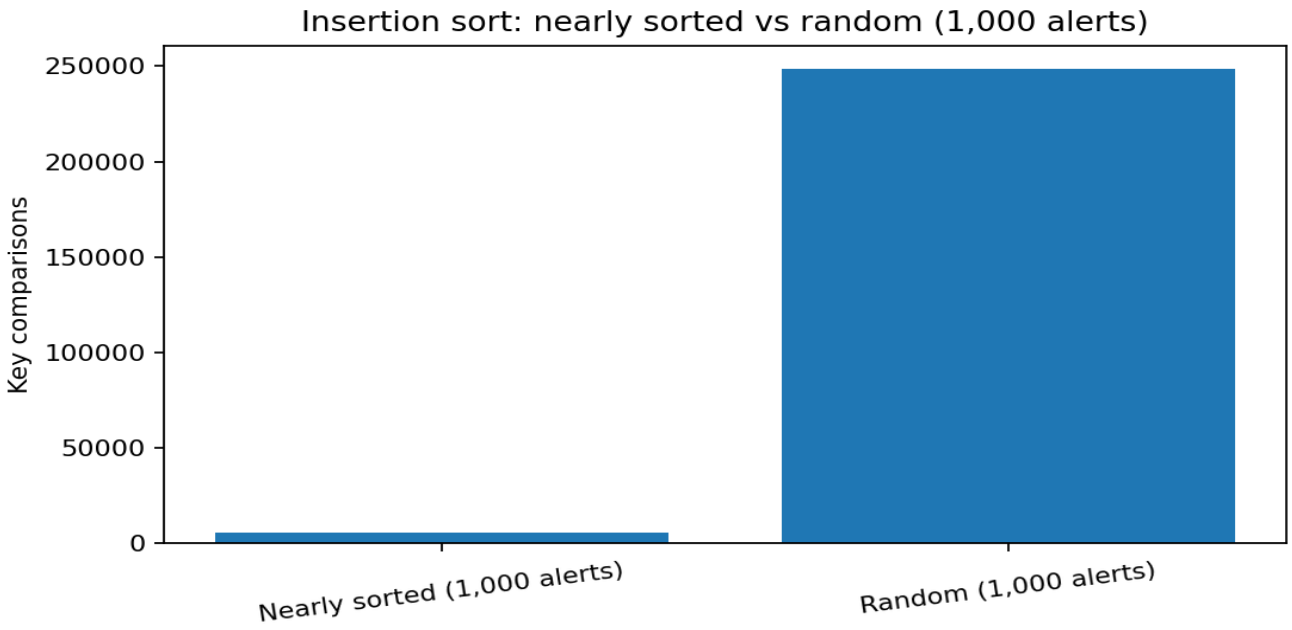


Interpretation: brute force grows quadratically. On this fixed sample, the x-sweep prunes most pairs after a short x-distance check. The exact count depends on point distribution, and adversarial or highly clustered data can reduce the benefit.

Insertion-sort comparisons

Input (1,000 alerts)	Key comparisons
Nearly sorted (1,000 alerts)	5,675
Random (1,000 alerts)	248,414

GRAPH:



The nearly sorted sample starts in descending severity order and then receives ten random swaps. It requires far fewer comparisons than the random input. This illustrates why insertion sort can be acceptable for tiny or nearly sorted inputs, but its quadratic worst case makes it a poor default for 50,000 alerts. Python's stable adaptive sort is the recommended production choice.

Operation-count derivations

- **Closest-pair brute force:** number of unique pair evaluations = $n(n-1)/2$. At n = 1,000: 499,500; at n = 4,000: 7,998,000; at n = 8,000: 31,996,000.
- **Sequential search:** up to b string comparisons per query; q queries can cost O(qb). Converting once to a set costs O(b) expected and each subsequent lookup is O(1) average.
- **Brute-force signature search:** up to $(n-m+1)m$ character comparisons. KMP builds the LPS table and scans the log in O(n+m), avoiding repeated comparisons after mismatch.

- **Insertion sort:** nearly sorted input can approach n−1 comparisons; random input has $\Theta(n^2)$ comparisons. Built-in Timsort has O(n log n) worst-case comparison complexity and exploits existing order.

14. Optimisation Decisions and Production Notes

- **Sorting:** use Python's built-in stable sort for 50,000 alerts. Equal severity alerts preserve their relative input order.
- **Blocklist:** create a set once after loading or refreshing the blocklist. Do not rebuild the set for each query.
- **Signature search:** KMP is linear in log length plus pattern length and supports overlapping matches.
- **Closest pair:** use squared distances inside the loop, take one square root for the final answer, skip self/reversed pairs, and break once dx^2 is at least the best squared distance.
- **Coordinates:** if locations are latitude/longitude rather than planar x/y kilometres, Euclidean distance is not sufficient without a coordinate projection or geodesic distance. This assignment explicitly labels the sample as (x, y) in km.
- **Ties:** the closest-pair implementation returns the first best pair encountered by its sorted traversal if multiple pairs have identical minimum distance. Tests should verify the distance and validity of the pair when a tie exists.

16. Conclusion

The original SentinelOps module contained nine logic defects across sorting, sequential search, brute-force string matching and closest-pair computation. The defects caused alert corruption, false blocklist results, missed and false signature hits, and selection of a farthest pair instead of a closest pair. Each defect was corrected with a targeted change and verified using supplied and boundary test cases.

The optimised implementation uses stable Timsort for alert ordering, a reusable set for blocklist membership, KMP for signature matching, and x-gap pruning with squared distances for closest-pair calculations. Experimental results show large reductions in closest-pair distance computations for the tested datasets, while the complexity discussion states the pruning method's quadratic worst case honestly. The supplied test harness provides repeatable checks before repository submission.

Appendix A – Reproducible Experimental Results

Batch size	Brute-force count	Pruned count	Reduction
1,000	499,500	1,778	280.9×
4,000	7,998,000	10,304	776.2×
8,000	31,996,000	11,704	2,733.8×

Dataset recipe: random.Random(42); for each batch size, generate points (random() × 10,000, random() × 10,000). Generate a new dataset for each batch size from the continuing seeded random generator. Counts above are the recorded results for this recipe.

Appendix B – Academic Integrity and Final Checklist

- Fill in student name and register number on the title page.
- Run the optimised program and test harness locally.
- Capture real screenshots of source code and terminal execution.
- Include the screenshot files in the Report folder.
- Verify the GitHub repository structure and push all required files.
- Submit only the required repository/report items according to the instructor's direction.